Perplexity for Instruction-Trained Models: why the standard benchmark cannot measure them, and what quantisation actually costs

Dmitry "Leo" Kuznetsov

September 2, 2026

Abstract

Perplexity is the default way to compare quantisations of a language model, but the standard tool cannot measure an instruction-trained model at all. It scores raw prose, and a model finetuned with RL to follow instructions reads raw prose as something to answer rather than continue. We measure one such model at perplexity 507 on WikiText-2 and 8.46 on the same class of text once that text is wrapped in the model's own chat template, a factor of 60. We describe pplit, a tool that renders each prompt through the template stored in the model's own GGUF and scores only the response tokens, and use it for two questions the standard benchmark answers badly. First, what a vendor's shipped 4-bit release actually costs relative to the weights it came from: for quantisation-aware-trained releases the answer is that the two are simply different models, worse by 4.72 percent at one size, better by 3.07 percent at another, with nothing about size or architecture predicting which. Second, what ordinary post-training quantisation costs on instruction-following text, where we find the standard raw-text benchmark can understate the damage by a factor of three, and that the penalty grows with conversation depth.

1 Why the standard benchmark does not measure these models

llama-perplexity tokenizes a text file, splits it into n_ctx chunks and scores the second half of each. That protocol is sound and it is what almost every published perplexity number for a GGUF model comes from. It is also useless for an instruction-trained model, and the size of the failure is easy to miss because the tool returns a number rather than an error.

We measured nine instruction-trained models three ways: the standard WikiText-2 benchmark, the response text of our own corpus with the chat framing stripped, and the same responses scored inside the model's own chat template. All llama-perplexity runs use -c 512 and --chunks 60.

Eight of the nine models are overstated by 2x to 8x, which is bad but survivable: a reader could learn to treat the standard benchmark as a biased ruler. The dense gemma-4 12B is overstated by a factor of 60. A number like 507 does not read as a protocol error. It reads as a broken model, and that is what makes the standard tool unusable here rather than merely inaccurate.

Two effects stack, and the middle column separates them. Register costs a factor of roughly 4 to 5: encyclopedia prose against assistant prose, both unframed. Framing costs a further 1.2x to 1.6x on eight models and 9.26x on the dense 12B. The gemma dense line requires its chat frame. The E-series and all four Qwen models merely benefit from it.

model	quant	published by	WikiText-2	raw replies	chat-framed	ratio
gemma-4 E2B	q4_0 QAT	Google	46.00	9.88	6.4465	7.1x
gemma-4 E4B	q4_0 QAT	Google	29.91	7.70	5.1417	5.8x
gemma-4 E2B	Q4_K_M PTQ	third party	142.42	27.99	17.2175	8.3x
gemma-4 E4B	Q4_K_M PTQ	third party	56.31	12.13	9.7072	5.8x
gemma-4 12B	Q4_K_M PTQ	third party	507.00	78.38	8.4623	59.9x
Qwen3.5 0.8B	Q4_K_M PTQ	third party	18.17	9.28	6.8417	2.7x
Qwen3.5 2B	Q4_K_M PTQ	third party	12.12	6.81	5.4274	2.2x
Qwen3.5 4B	Q4_K_M PTQ	third party	9.31	5.37	4.4940	2.1x
Qwen3.5 9B	Q4_K_M PTQ	third party	7.76	4.71	3.9347	2.0x

Table 1: Nine instruction-trained models scored three ways. Google-published and third-party quantisations are separated; the protocol failure in the right column is a property of the model and the framing, not of who quantised it.

2 What pplit does

pplit reads prompt and response pairs, renders each prompt through the jinja chat template stored in that model's own GGUF, and scores only the response tokens. Position i's logits predict token $i+1$, so scoring the response requires logit rows from one position before the reply starts. Everything earlier is prompt and is not scored.

Rendering per model at run time rather than freezing token ids is not a convenience. The gemma-4 E-series and the dense sizes ship different templates: a 12B prompt ends with an empty thought-channel block that an E-series prompt does not have. Scoring the 12B through the E-series template moves it from 7.67 to 28.30. A corpus of frozen token ids can only ever be correctly framed for one family, and the failure presents as a bad model rather than a bad prompt.

The tool reports five numbers per run rather than one, and that choice is not hedging. Perplexity is the exponential of the mean negative log probability of the corpus token, so it is sensitive to how much probability mass reached the right answer. Top-1, top-5 and top-10 are sensitive only to ranking. The rank percentiles say where the corpus token sat in the model's ordering of the whole vocabulary. The five disagree, and the disagreement is usually the interesting part.

2.1 Why one number is not enough

Veličković et al. give the theoretical reason [1]. They prove that for a compact decoder-only Transformer, confident accurate prediction of one sequence forces the existence of another sequence with low perplexity that is predicted incorrectly, and they show by iso-perplexity analysis that perplexity will not prefer the more accurate of two models unless the confidence gain happens to line up with the accuracy gain. Perplexity measures how much probability mass landed on the observed token. That is a different question from whether the model would pick the right token.

Our data contains a clean instance of the failure they describe. In section 4, the 31B shipped 4-bit file has better perplexity than the full-precision weights it came from, 8.3109 against 8.5744, while being worse on every ranking metric at once: top-1 57.36 against 61.24 percent, top-5 80.27 against 84.75, top-10 86.71 against 89.41, and rank p90 16 against 11. Confidence went up and accuracy went down, which is exactly the case in which perplexity picks the wrong winner. A study reporting perplexity alone would have concluded that the 4-bit file was the better model.

This is why every table here carries the ranking metrics alongside the perplexity, and why section 4 also reports KL divergence. Three different questions are being asked. Perplexity asks how much probability reached the right token. The rank statistics ask whether the model would have chosen it. KL asks how far the whole distribution moved. A quantisation can leave one almost untouched while moving another a great deal, and which of them matters depends on what the model is for: a sampler cares about the distribution, a greedy decoder cares about the ranking, and a scorer cares about the probability.

2.2 KL divergence without a teacher-sized dump

Perplexity measures agreement with a corpus. It cannot answer the question this paper is about, which is how far a quantisation moved from the weights it was derived from. For that pplit has an opt-in mode: one pass over the reference model writes its distribution per scored row, and a second pass scores a student against it.

Storing every logit is impractical at gemma's vocabulary of 262144, which is about 1 MB per token in f32 and roughly 26 GB for our corpus. We store the reference's top-64 log probabilities per row plus the remaining probability mass as a single lumped bucket, which is about 13 MB. Lumping the tail makes the reported divergence a lower bound on the true value by the data processing inequality, tightening as K grows. The dump header carries the vocabulary size and row count, so a student that tokenizes differently is reported as a mismatch rather than silently compared against misaligned rows.

Two sanity results. A model scored against its own dump reports KL 0.00000 and 100.00% argmax agreement. A Q4_K_M scored against the Q8_0 of the same file reports KL 0.04227 at 88.80% agreement, while its perplexity moves only 0.7%. One token in nine changes its top pick for a perplexity difference most readers would call noise.

3 Setup

Three corpora are used. IFBench, 100 prompt and response pairs, single-turn. StructFlowBench in two forms, a single-turn corpus of first turns and a multi-turn corpus of 40 conversations totalling 114 turns, which lets turn depth be varied with the source held fixed. The main corpus is IFBench unless stated otherwise, 329 KB. Responses are model written, which is deliberate: text that no model under test produced, so the number is a perplexity rather than the roughly 1.3 a model scores against its own output. Every run scores 24958 target tokens for gemma and 24960 for Qwen, 100 items, at most 320 response tokens per item, context 8192, one model resident at a time.

Two hosts: a Metal host and an AMD host. The AMD host runs the Vulkan backend unless stated otherwise. We report no timings anywhere in this paper, so the machines matter only as independent implementations of the same arithmetic. BF16 files are local convert_hf_to_gguf.py conversions of Google's -qat-q4_0-unquantized checkpoints, which is what makes the comparison in section 4 controlled: the shipped q4_0 and our BF16 are the same checkpoint, not two different training runs.

Provenance is kept strict throughout. Google's QAT releases derive from a separate checkpoint to the base instruction-tuned release, so a QAT file and a third party's quantisation of the base weights differ by a training run as well as by a quantiser. Every gemma comparison in section 4 is therefore between Google's own shipped file and a BF16 conversion of the same Google checkpoint.

Third-party quantisations are reported separately and labelled as such, and are never used as a reference for a Google file.

4 The shipped 4-bit file is a different model, not a lossy copy

Each shipped file against a BF16 conversion of its own checkpoint. Same corpus, same engine, same 24958 targets, identical tensor names, and F32 norms in both files. Only the weight matrices differ.

size	full precision	shipped 4-bit	difference	KL	argmax agree
E2B	6.2730	6.4478	4-bit worse by 2.79%	0.03867	91.53%
E4B	5.1569	5.1457	same within noise	0.02504	93.04%
12B	7.2460	7.5881	4-bit worse by 4.72%	0.02786	93.77%
26B-A4B	7.0570	7.0651	same within noise	0.05049	91.73%
31B	8.5744	8.3109	4-bit better by 3.07%	0.03687	92.37%

Table 2: Google's own shipped q4_0 against a BF16 conversion of the same Google QAT checkpoint. Both files in every row come from the model's author.

It is natural to read the left column as the real model and the middle column as a lossy copy of it. For these files that reading is backwards.

Quantisation-aware training runs the 4-bit conversion inside the training loop, so the function being optimised is the 4-bit one. The full-precision weights are scaffolding left over from that process, not a version anyone was meant to run. Once that is clear, there is no reason the 4-bit file has to be worse, and it is not: it is worse at two sizes, better at one, and indistinguishable at two.

The spread runs from 4.72% worse to 3.07% better, and nothing about a model's size or architecture predicts where in that range it will land. The 31B is the clearest case. Its 4-bit file does not merely edge ahead on perplexity, it wins on all five metrics at once, which is not what a rounding error looks like.

The KL column says these are genuinely different models in every row: the two files disagree about the single most likely next token between 6 and 9 percent of the time. What KL does not do is predict the direction. The 26B pair is the most distant of the five by KL and the closest by perplexity.

The practical consequence is short. Do not treat a vendor's 4-bit release as a degraded copy of its full-precision weights, and do not infer one from the other. They are different models. Measure the one you intend to ship.

For scale, the same experiment on a family with no QAT anywhere in its history. Qwen publishes no quantisations of its own models, so every Qwen figure in this paper is necessarily a third-party quantisation. That is a difference in kind from the gemma table above, where the quantised file and the checkpoint both come from the model's author, and it should be held in mind when reading the two side by side. What is preserved is the structure of the comparison: one checkpoint, two encodings of it, with the BF16 converted by us from Qwen's own published weights.

Every column is monotonic in model size. The perplexity cost falls, the distributional distance falls, and argmax agreement rises, all three together, which is what one would expect if larger

size	BF16	UD-Q4_K_XL	ppl cost	KL	argmax agreement
2B	5.3644	5.4421	+1.45%	0.02174	92.07%
4B	4.4629	4.5086	+1.02%	0.01828	93.55%
9B	3.9271	3.9525	+0.65%	0.01465	94.29%

Table 3: Third-party post-training K-quants of Qwen3.5 against BF16 conversions of Qwen's own checkpoints. Qwen publishes no quantised files, so no vendor-published comparison is possible for this family.

models have more redundancy to absorb rounding error. Ordinary post-training quantisation on this family is orderly and it gets cheaper as the model grows.

The gemma column is neither. It is not monotonic, it does not get cheaper with size, and at two of its three sizes it costs more than the generic K-quant does on a comparable model. We are not claiming the two quantisers are interchangeable. The formats differ, the families differ, the training differs, and one column is vendor-published where the other cannot be. We are claiming something narrower: a file whose entire purpose is to be the native 4-bit encoding of weights trained for 4-bit does not behave better than a general-purpose quantiser applied to weights that were never prepared for it, and it behaves less predictably across sizes.

Note also that the KL columns are the more orderly of the two measures here. The Qwen KLs fall smoothly from 0.0217 to 0.0147 while gemma's sit between 0.0250 and 0.0387 with no relation to the perplexity costs beside them. If one had to pick a single number for "how much did this quantisation move the model", KL behaves more like a physical quantity than the perplexity delta does.

5 What ordinary quantisation costs, and where the standard benchmark misses it

The previous section covered vendor releases whose weights were trained through the quantiser. The commoner case is a plain post-training quantisation of an ordinary instruction-trained checkpoint, and there the question is simply how much accuracy it costs.

We took Qwen's own published weights for two sizes, converted them to BF16 ourselves, and quantised those BF16 files to Q4_K_M ourselves. The checkpoint is therefore identical on both sides and the only variable is the quantiser. Each pair was then scored three ways: the standard raw-text benchmark, single-turn instruction-following text, and multi-turn conversations where later turns are scored with the earlier ones as context.

Three things come out of this.

The standard benchmark can badly understate the cost. On the 0.8B it reports 2.10 percent where instruction-following text shows 6.9 to 8.5 percent, a factor of three to four. On the 2B the two roughly agree. So the raw-text number is not a conservative proxy that happens to be a little low, it is a number whose relationship to the quantity of interest varies by model. If you care how a quantisation behaves in a chat application, the raw-text benchmark cannot tell you, and it will not warn you when it is wrong.

Depth makes it worse. Comparing like with like, single-turn against multi-turn on the same corpus,

model	corpus	BF16	Q4_K_M	cost
Qwen3.5-0.8B	WikiText-2, raw	15.5878	15.9150	+2.10%
Qwen3.5-0.8B	IFBench, single-turn	6.7184	7.2428	+7.80%
Qwen3.5-0.8B	StructFlow, single-turn	5.4940	5.8743	+6.92%
Qwen3.5-0.8B	StructFlow, multi-turn	4.7699	5.1759	+8.51%
Qwen3.5-2B	WikiText-2, raw	10.7872	11.3778	+5.47%
Qwen3.5-2B	IFBench, single-turn	5.3644	5.6438	+5.21%
Qwen3.5-2B	StructFlow, single-turn	4.9253	5.0849	+3.24%
Qwen3.5-2B	StructFlow, multi-turn	4.3230	4.4845	+3.74%
Qwen3.8-27B	WikiText-2, raw	6.3530	6.3877	+0.55%
Qwen3.8-27B	IFBench, single-turn	5.6195	5.2050	-7.38%
Qwen3.8-27B	StructFlow, single-turn	4.0234	3.9551	-1.70%
Qwen3.8-27B	StructFlow, multi-turn	3.3293	3.2652	-1.93%

Table 4: Post-training quantisation of one checkpoint, measured three ways. Both files in every row derive from the same published weights.

the penalty rises on both models: 6.92 to 8.51 percent on the 0.8B and 3.24 to 3.74 percent on the 2B. Note that multi-turn perplexity is lower in absolute terms, because a model with more context predicts better, and yet the quantisation penalty on it is larger. A single-turn measurement therefore flatters a quantisation that will be deployed in a conversation.

The quantiser recipe matters more than the bit width, and the standard benchmark cannot see the difference. On the 27B we compared two 4-bit files built from the same checkpoint: our own stock Q4_K_M and a third party's UD-Q4_K_M.

Qwen3.8-27B	WikiText-2	IFBench framed	KL vs BF16	argmax agree
BF16 (reference)	6.3530	5.6195	n/a	n/a
our Q4_K_M	6.3877	5.2050	0.03194	92.61%
UD-Q4_K_M	6.3857	5.8820	0.01697	94.48%
Q8_0	n/a	5.6074	0.00403	96.94%

Table 5: Two 4-bit encodings of one checkpoint. WikiText-2 puts them 0.03 percent apart; chat-framed text puts them 13 percent apart.

On WikiText-2 the two 4-bit files are 6.3877 and 6.3857, which is 0.03 percent apart and well inside any reasonable noise. On chat-framed text they are 5.2050 and 5.8820, 13 percent apart. The standard benchmark does not merely understate the difference between these two quantisations, it cannot see it.

Which of the two is better depends on the question. Perplexity prefers ours. KL says the other is nearly twice as faithful to the weights it came from, 0.017 against 0.032, with argmax agreement 94.5 against 92.6 percent. Both statements are true; they are answers to different questions, which is the point of section 2.1. The Q8_0 row is the control: lowest divergence, highest agreement, perplexity nearest the reference, which is what a near-lossless encoding must look like and confirms the measurement is working.

One consequence for anyone benchmarking quantisations by KL against a Q8_0 reference, which is common practice [2]. Q8_0's own divergence from BF16 is 0.00403 here. Against a 2-bit quant, where divergences run above 0.1, that is negligible. Against a good 4-bit quant at 0.017 it is roughly

a quarter of the signal, so the choice of reference starts to matter exactly where 4-bit comparisons are decided.

Sign is not guaranteed. At 27B the 4-bit file has better perplexity than the BF16 it came from, on all three chat-framed corpora, by 1.7 to 7.4 percent. This is the same pattern as the vendor round trips of section 4, and the same warning applies: a quantisation that scores better on a corpus has not been shown to be a better model, only a different one that suits this corpus.

Finally, the framing effect that motivates the whole tool is not universal. Qwen3.8-27B reads 6.3530 raw against 5.6195 framed, a ratio of 1.13, the lowest we measured. It is a gemma-family property and it does not grow with size across vendors: a 27B Qwen shows less of it than a 2B Qwen.

6 What the measurement is sensitive to

6.1 The backend is part of the measurement

We ran the same file, on the same host, over the same 25 items, changing only the compute backend. Item sampling noise cancels exactly in a paired design like this, so the spread is backend arithmetic and nothing else. We use AMD ROCm and Vulkan as the two GPU references, with CPU as a third implementation.

backend	ppl	vs Vulkan
Vulkan	8.2591	n/a
CPU	8.2895	+0.37%
AMD ROCm	8.3439	+1.03%

Table 6: Same file, same host, same 25 items, changing only the compute backend.

A 1.03% spread between ROCm and Vulkan is the same order of magnitude as the quantisation costs in section 4. The variance is also not uniform: the same comparison on E2B q4_0 across Metal and Vulkan agrees to 0.02%, and 12B BF16 agrees to 0.07%. It is the quantised 12B specifically that is backend sensitive. A quantisation cost must therefore be quoted with its backend as well as its corpus, and cross-host comparisons of quantised models need the backend held fixed or measured. All gemma and Qwen figures elsewhere in this paper come from the Vulkan backend, so they are internally consistent even where the absolute values would shift on another one.

6.2 The noise floor is larger than most reported effects

Companion work on these corpora measured the paired-delta sampling floor directly, by re-running the same corpus and the same two files at 410 items instead of 100. The delta moved by about 0.45 percentage points. The reason is that the effective sample size for a paired log-loss difference is the number of items, not the number of tokens: per-token log loss is strongly correlated inside a single response, so 320 tokens of one answer is closer to one observation than to 320.

E4B's -0.22% in section 4 is inside that floor and should be read as "no measurable cost", not as "zero cost". E2B and the 12B are outside it.

7 What these numbers do not say

The metric scores agreement with whoever wrote the corpus. Our responses are model written, so a model whose style resembles the corpus author's is flattered, and cross-family comparisons inherit that bias. Companion work measured this rather than merely arguing it, using two human-authored corpora: physician-written HealthBench responses and pre-ChatGPT Reddit debate. Both were used solely for research and validation, neither is redistributed, and no text from either appears here. Both land 2.8x and 5.7x worse than the model-authored corpora, while two model-authored corpora spanning a 3x length range agree within 3%. The common factor across the two hard corpora is who wrote the text, not length and not domain.

That result bounds this paper. Within one checkpoint, comparing two quantisations of it, the instrument is doing what it was built for and the ordering is stable everywhere it has been tested. Across model families it is measuring proximity to a corpus author as much as anything else.

Quantisation cost is also not transferable between corpora. Companion work repeated one BF16 against q4_0 comparison on four corpora and got +5.79%, +5.16%, +4.87% and +3.03%, ordering inversely with corpus difficulty. The direction survives everywhere. The number does not travel.

8 Reproducing this

Everything needed is in the repository: the tool, the corpus, and a llama.cpp submodule pinned to the commit these numbers were produced with. The two human-authored corpora cannot be redistributed and ship as rebuild scripts that fetch from the canonical source.

```
cmake -B build -DGGML_VULKAN=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build --target pplit
./build/pplit -m ref-BF16.gguf -f pplit-corpus.txt -ngl 99 --kld-save ref.kld
./build/pplit -m shipped-q4_0.gguf -f pplit-corpus.txt -ngl 99 --kld ref.kld
```

References

[1] P. Veličković, F. Barbero, C. Perivolaropoulos, S. Osindero, and R. Pascanu. Perplexity cannot always tell right from wrong. arXiv:2601.22950.

[2] Independent KLD benchmark of Qwen3.8-27B quantisations against a Q8_0 reference. https://huggingface.co/unsloth/Qwen3.8-27B-GGUF/discussions/49